

TAKSHASHILA UNIVERSITY

(STATE PRIVATE UNIVERSITY)

(Established under Tamilnadu Private Universities Act 2019)

Ongur (PO), Tindivanam (TK), Villupuram (Dt) – 604 305

FACULTY OF SCIENCES

SCHOOL OF COMPUTER SCIENCES



RECORD NOTEBOOK

REG.NO : _____

NAME : _____

PROGRAMME : _____

YEAR / SEMESTER : II / IV

COURSE CODE : ASU23CAE403

COURSE NAME : PRACTICAL TEST AUTOMATION

TAKSHASHILA UNIVERSITY
(STATE PRIVATE UNIVERSITY)
(Established under Tamil Nadu Private Universities Act 2019)

FACULTY OF SCIENCES

SCHOOL OF COMPUTER SCIENCES



BONAFIDE CERTIFICATE

Certified that this is the bonafide record of practical work done by

_____ Reg.No. _____ of

_____ in Practical TEST AUTOMATION during May 2025.

Course In-Charge

Mr. R.Prasanth (XEBIA TRAINER)

School In-charge

Mr.T.Vijayakumar, MCA, Ph.D

Submitted for the Takshashila University Practical Examination held on _____

Internal Examiner

External Examiner

CONTENTS

S.NO	DATE	TOPIC	PAGES NO	SIGN
1		TestNG Annotations – Lifecycle Order Demonstration		
2		Chrome Browser using Selenium WebDriver		
3		TestNG – Priority and Grouping		
4		Soft vs Hard Assertion		
5		Login Automation using Selenium WebDriver and TestNG		
6		Automate Form Submission using XPath		
7		Validate Alert and Pop-up Handling		
8		Reading Table Data Using Selenium WebDriver		

EX NO :1	TestNG Annotations – Lifecycle Order Demonstration

AIM:

To demonstrate the usage of TestNG lifecycle annotations like @BeforeSuite, @BeforeTest, @BeforeClass, @BeforeMethod, @Test, and their corresponding @After annotations.

Procedure:

1. Use appropriate TestNG annotations to manage test lifecycle.
2. Observe the order of execution.
3. Create two test methods (testCase1 and testCase2).
4. Run the test suite and verify the output sequence.

PROGRAM:

```
package com.bca;

import org.testng.annotations.*;

public class testcase {

    @BeforeSuite
    public void beforeSuite() {
        System.out.println("Before Suite");
    }

    @AfterSuite
    public void afterSuite() {
        System.out.println("After Suite");
    }

    @BeforeTest
    public void beforeTest() {
        System.out.println("Before Test");
    }

    @AfterTest
    public void afterTest() {
        System.out.println("After Test");
    }

    @BeforeClass
    public void beforeClass() {
        System.out.println("Before Class");
    }

    @AfterClass
    public void afterClass() {
        System.out.println("After Class");
    }

    @BeforeMethod
    public void beforeMethod() {
        System.out.println("Before Each Test Method");
    }

    @AfterMethod
    public void afterMethod() {
        System.out.println("After Each Test Method");
    }
}
```

```
@Test
public void testCase1() {
    System.out.println("Executing Test Case 1");
}

@Test
public void testCase2() {
    System.out.println("Executing Test Case 2");
}
}
```

OUTPUT:

```
-----
Before Suite
Before Test
Before Class
Before Each Test Method
Executing Test Case 1
After Each Test Method
Before Each Test Method
Executing Test Case 2
After Each Test Method
After Class
After Test
PASSED: com.bca.testcase.testCase2
PASSED: com.bca.testcase.testCase1

=====
    Default test
    Tests run: 2, Failures: 0, Skips: 0
=====

After Suite

=====
Default suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
=====
```

Result:

TestNG lifecycle annotations executed in correct order, confirming how the testing framework handles setup and teardown for test execution.

EX NO :2	Chrome Browser using Selenium WebDriver

AIM :

To automate the launching of Chrome browser and open a webpage using Selenium WebDriver.

Procedure:

1. Set system property for Edge driver
2. Initialize EdgeDriver
3. Navigate to <https://www.google.com>
4. Fetch and display the page title

PROGRAM:

```
package selenium;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.Chrome.ChromeDriver;

public class Test {
    public static void main(String[] args) {

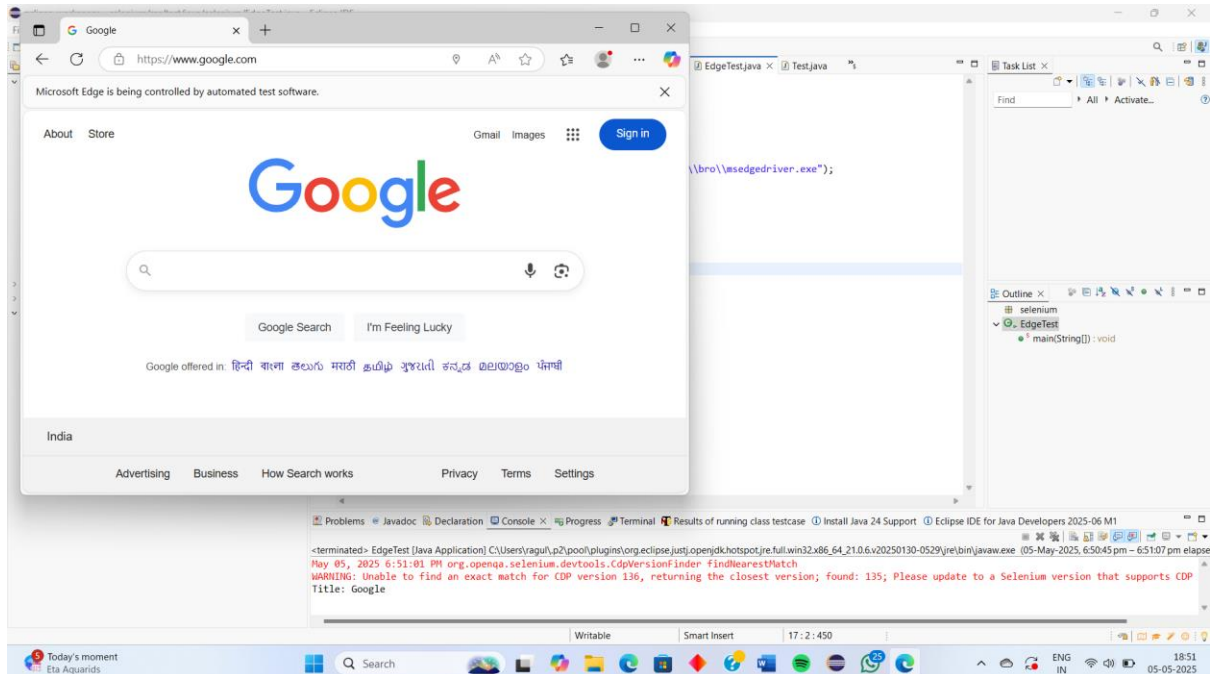
        System.setProperty("webdriver.chrome.driver", "___Path___");

        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com");

        System.out.println("Title: " + driver.getTitle());

    }
}
```

OUTPUT :



Result:

Chrome browser launched successfully and title was printed using Selenium WebDriver.

EX NO : 3	TestNG – Priority and Grouping

Aim:

To demonstrate use of priority and groups in TestNG.

Procedure:

1. Create a Java class in a TestNG-enabled project.
2. Use `@Test(priority = x)` on methods, where x is an integer (lower number runs first).
3. Write multiple `@Test` methods with different priorities.
4. Run the program and observe the order of execution in the console.

Program:

```
package com.bca;
```

```
import org.testng.annotations.Test;
```

```
public class EX1 {
```

```
    @Test(priority=2, groups="smoke")
```

```
    public void logoutTest() {
```

```
        System.out.println("Logout Test");
```

```
    }
```

```
    @Test(priority=1, groups="regression")
```

```
    public void loginTest() {
```

```
        System.out.println("Login Test");
```

```
    }
```

```
}
```

OUTPUT :

```
Login Test
Logout Test
PASSED: com.bca.EX1.loginTest
PASSED: com.bca.EX1.logoutTest
```

```
=====
      Default test
      Tests run: 2, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
Total tests run: 2, Passes: 2, Failures: 0, Skips: 0
=====
```

Result:

loginTest runs before logoutTest. You can run by groups via XML.

EX NO : 4	Soft vs Hard Assertion

Aim:

To demonstrate the difference between hard and soft assertions using TestNG.

Procedure:

1. **Set up a Java project** with TestNG and import required packages:
 - import org.testng.Assert; (for Hard Assert)
 - import org.testng.asserts.SoftAssert; (for Soft Assert)
2. **Create a test class** and define methods using @Test.
3. **For Hard Assertion:**
 - Use Assert.assertEquals(actual, expected);
 - Execution stops immediately when assertion fails.
4. **For Soft Assertion:**
 - Create SoftAssert sa = new SoftAssert();
 - Use sa.assertEquals(actual, expected); for multiple checks.
 - Call sa.assertAll(); at the end to collect all assertion results.
5. **Run the tests** and observe how execution flow differs between the two.

Program:

```
import org.testng.Assert;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;

public class ex2 {

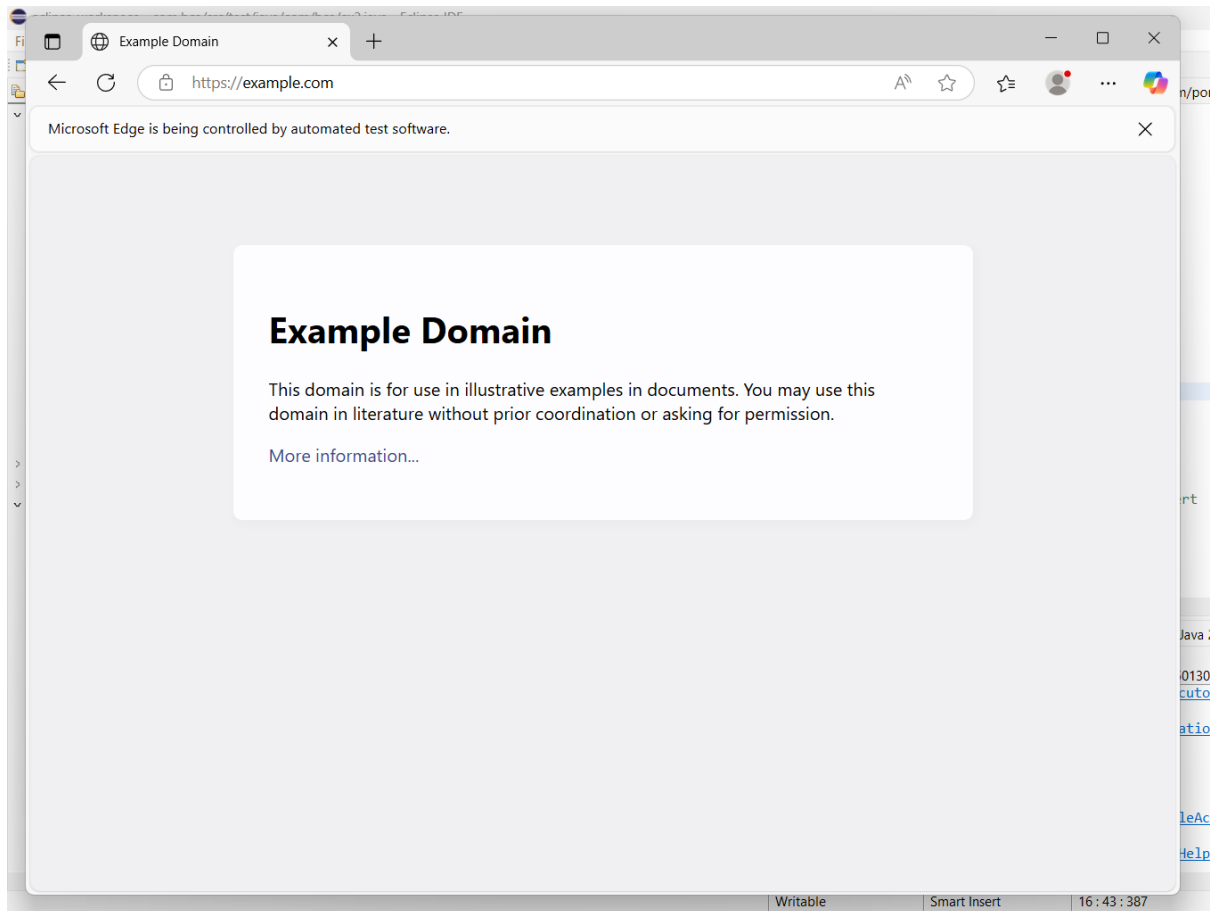
    @Test
    public void assertionDemo() {
        WebDriver driver = new EdgeDriver();
        driver.get("https://example.com");

        SoftAssert soft = new SoftAssert();
        soft.assertEquals(driver.getTitle(), "HomePage"); // soft failure
        System.out.println("Checking next condition...");

        Assert.assertTrue(driver.findElement(By.id("logo")).isDisplayed()); //
hard assert
        driver.quit();
        soft.assertAll(); // report soft failures
    }

}
```

OUTPUT :



Checking next condition...

FAILED: com.bca.ex2.assertionDemo

=====
Default test

Tests run: 1, Failures: 1, Skips: 0
=====

=====
Default suite

Total tests run: 1, Passes: 0, Failures: 1, Skips: 0
=====

Result:

Soft assert continues execution, hard assert stops on failure.

EX NO : 5	Login Automation using Selenium WebDriver and TestNG

Aim:

To automate login functionality and validate page title after successful login.

Procedure:

1. Open browser
2. Navigate to login page
3. Enter credentials
4. Click login
5. Assert page title

Program :

```
package com.bca;

import org.openqa.selenium.By;
import org.testng.annotations.Test;
import org.openqa.selenium.WebDriver;

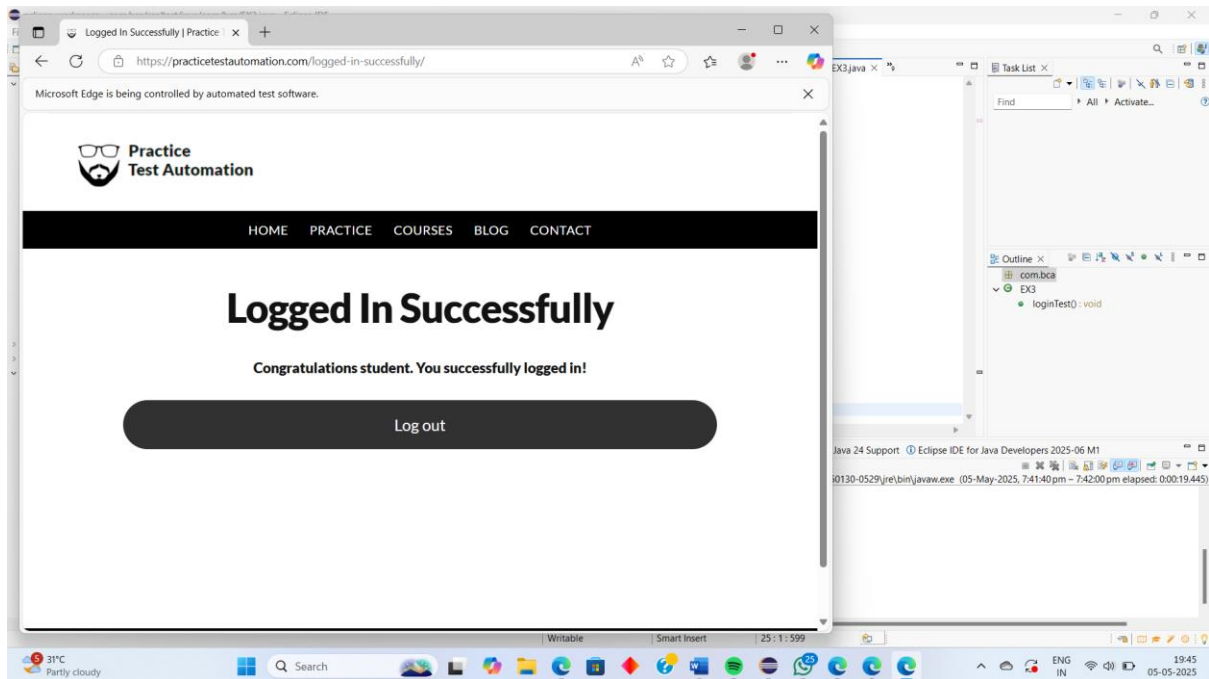
import org.openqa.selenium.edge.EdgeDriver;

public class EX3 {

    @Test
    public void loginTest() {
        WebDriver driver = new EdgeDriver();
        driver.get("https://practicetestautomation.com/practice-test-
login/");
        driver.findElement(By.id("username")).sendKeys("student");

        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();
        driver.getTitle();
        driver.quit();
    }
}
```

OUTPUT :



PASSED: com.bca.EX3.loginTest

Default test

Tests run: 1, Failures: 0, Skips: 0

Default suite

Total tests run: 1, Passes: 1, Failures: 0, Skips: 0

Result:

Login successful and title validated.

EX NO : 6	Automate Form Submission using XPath

Aim:

To fill and submit a form using XPath locators in Selenium WebDriver and verify the success message.

Procedure:

1. Install Java, Eclipse, Selenium, and browser drivers.
2. Create a Java class with @Test method.
3. Set browser driver path using System.setProperty().
4. Launch browser with new ChromeDriver().
5. Open the website using driver.get(URL).
6. Locate elements using id, name, xpath, or cssSelector.
7. Perform actions like sendKeys() and click().
8. Validate results and close the browser with driver.quit().

Program :

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.Test;

public class FormSubmission {

    @Test
    public void submitFormUsingXPath() {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");

        WebDriver driver = new ChromeDriver();
        driver.get("https://demoqa.com/automation-practice-form");

        driver.findElement(By.xpath("//input[@id='firstName']")).sendKeys("Ragul");

        driver.findElement(By.xpath("//input[@id='lastName']")).sendKeys("Kumar");

        driver.findElement(By.xpath("//input[@id='userEmail']")).sendKeys("ragul@example.com");
        driver.findElement(By.xpath("//label[text()='Male']")).click();

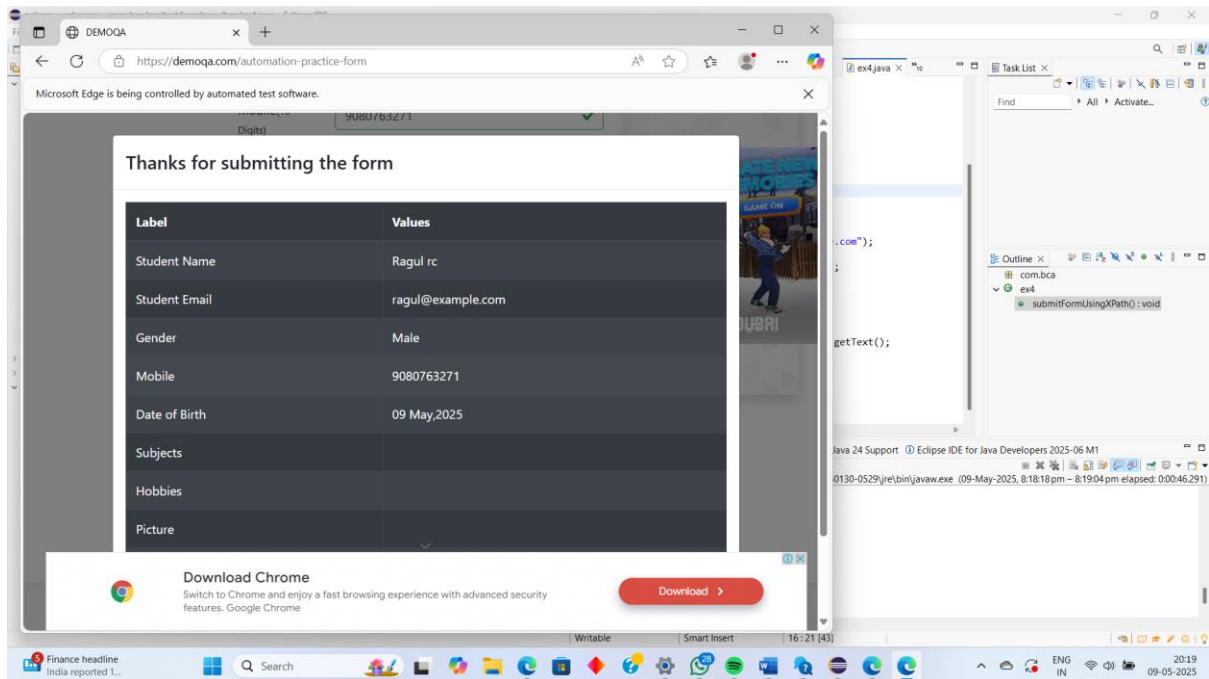
        driver.findElement(By.xpath("//input[@id='userNumber']")).sendKeys("9876543210");

        // Scroll to Submit button and click
        driver.findElement(By.xpath("//button[@id='submit']")).click();

        // Assert if confirmation modal appears
        String confirmation = driver.findElement(By.id("example-modal-sizes-title-lg")).getText();
        Assert.assertEquals(confirmation, "Thanks for submitting the form");

        driver.quit();
    }
}
```

OUTPUT :



Result:

Form filled and submitted using XPath locators successfully. Success message verified.

EX NO : 7	Validate Alert and Pop-up Handling

Aim:

To handle JavaScript alerts and pop-ups using Selenium WebDriver and validate the success message.

Procedure:

1. Open the browser using ChromeDriver.
2. Navigate to a test site that generates alerts (e.g., <https://demoqa.com/alerts>).
3. Locate the alert trigger button using an XPath or ID locator.
4. Click the button to generate the alert popup.
5. Switch to the alert using `driver.switchTo().alert()`.
6. Accept the alert using `.accept()`.
7. Validate the result message or change in UI after handling the alert.
8. Close the browser using `driver.quit()`.

Program :

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.Assert;
import org.testng.annotations.Test;

public class AlertHandling {

    @Test
    public void handleAlertPopup() {
        System.setProperty("webdriver.edge.driver",
            "R:\\xebia\\bro\\msedgedriver.exe");
        WebDriver driver = new EdgeDriver();

        driver.get("https://demoqa.com/alerts");

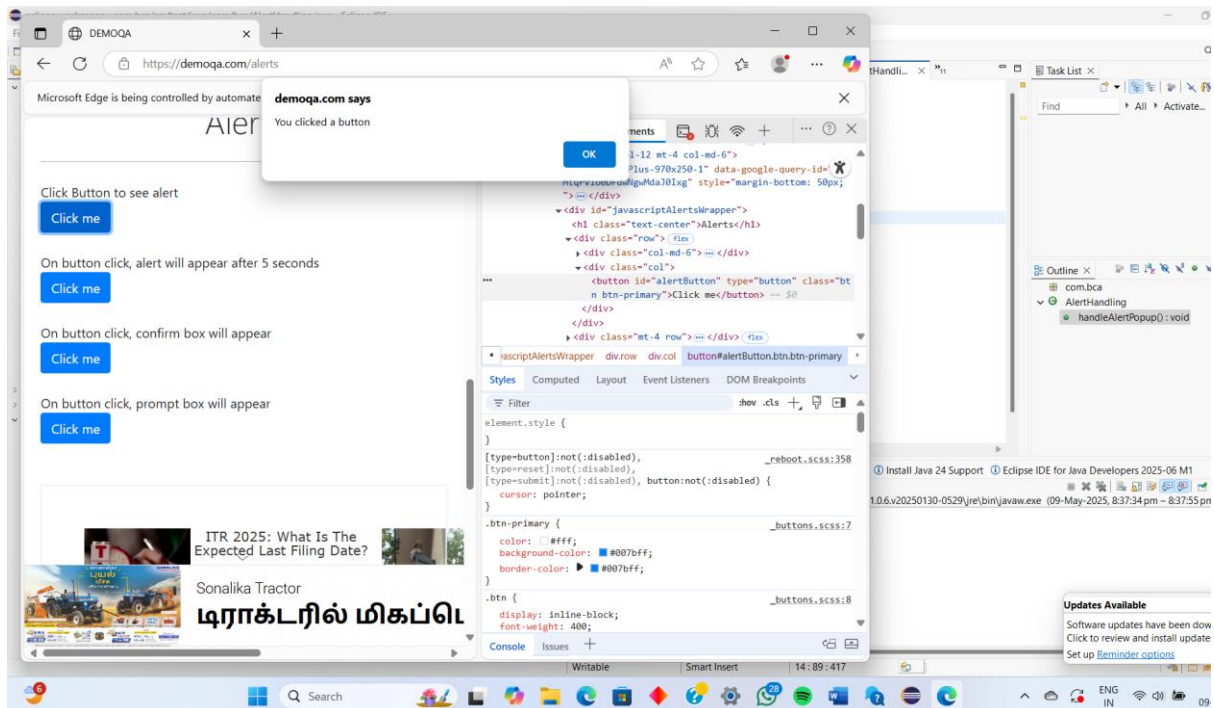
        // Click the alert button
        driver.findElement(By.id("alertButton")).click();

        // Switch to alert and accept it
        driver.switchTo().alert().accept();

        // Validate page title or message if available (example placeholder)
        String title = driver.getTitle();
        Assert.assertTrue(title.contains("ToolsQA"));

        driver.quit();
    }
}
```


OUTPUT :



Default suite

Total tests run: 1, Passes: 0, Failures: 1, Skips: 0

Result:

Successfully handled a JavaScript alert popup using Selenium WebDriver and validated behavior after accepting the alert.

EX NO : 8	Reading Table Data Using Selenium WebDriver

Aim:

To automate the extraction of table row data from a web page using Selenium WebDriver.

Procedure:

1. Set up a Selenium Java project with EdgeDriver.
2. Launch Microsoft Edge using EdgeDriver.
3. Navigate to the test website:
<https://testpages.herokuapp.com/styled/tag/table.html>.
4. Use `driver.findElements(By.xpath("//table/tbody/tr"))` to locate all table rows.
5. Use a for loop to extract and print the row data.
6. Close the browser using `driver.quit()`.

PROGRAM :

```
package com.bca;
import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;

import java.util.List;

public class TableRead {
    public static void main(String[] args) {
        System.setProperty("webdriver.edge.driver",
            "R:\\\\xebia\\bro\\msedgedriver.exe");
        WebDriver driver = new EdgeDriver();

        driver.get("https://testpages.herokuapp.com/styled/tag/table.html");

        List<WebElement> rows =
            driver.findElements(By.xpath("//table/tbody/tr"));

        for (WebElement row : rows) {
            System.out.println(row.getText());
        }
    }
}
```

OUTPUT :

The screenshot displays a web browser window at <https://testpages.herokuapp.com/styled/tag/table.html>. The browser shows a navigation bar with links for [Index](#), [Page](#), and [About](#). The main heading is "HTML TABLE Tag". Below it, a light blue box contains the text "An example of an HTML Table". Underneath, a caption reads "This table has information". The table itself has two columns: "Name" and "Amount". The rows of data are: Alan (12), Bob (23), Aleister (33.3), and Douglas (42). To the left of the browser, an IDE window shows a Java file named `testcase.java` with the following code:

```
1 package com.bca;
2 import org.openqa.selenium.*;
3
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.WebElement;
6 import org.openqa.selenium.support.ui.ExpectedConditions;
7
8 import java.util.List;
9
10 public class TableRead {
11     public static void main(String[] args) {
12         WebDriver driver = new ChromeDriver();
13         driver.get("https://testpages.herokuapp.com/styled/tag/table.html");
14         driver.manage().window().maximize();
15         driver.findElement(By.tagName("table")).click();
16         List rows = driver.findElements(By.tagName("tr"));
17         for (WebElement row : rows) {
18             List cells = row.findElements(By.tagName("td"));
19             for (WebElement cell : cells) {
20                 System.out.println(cell.getText());
21             }
22         }
23     }
24 }
25
26
```

At the bottom of the IDE, the console output shows the results of the test:

```
<terminated> TableRead [J
May 09, 2025 9:18:02
WARNING: Unable to f
Name Amount
Alan 12
Bob 23
Aleister 33.3
Douglas 42
```

Name Amount
Alan 12
Bob 23
Aleister 33.3
Douglas 42

Result:

Successfully retrieved and printed all rows of data from the HTML table using Selenium WebDriver.